UNIVERSIDADE PAULISTA

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Gustavo Alves Pigaiani - R215989
Marcos Vinícios Postal - H32DED0
Nicholas Kulicz Pazzotti - R4654G6

Projeto Integrado de Aplicação, Banco de Dados e API REST com SQL Server Express

CAMPINAS
2026

SUMÁRIO

1. INTRODUÇÃO.....4
1.1. Objetivo Geral..... 5
1.2. Objetivos Específicos..... 5
1.3. Escopo do Projeto..... 5
2. ARQUITETURA DO SISTEMA.....6
2.1. Camada de Apresentação - Frontend..... 7
2.2. Camada de Aplicação - Backend..... 8
2.3. Camada de Persistência - Banco de Dados..... 9
2.4. Integração entre as Camadas..... 9
3. SQL SERVER E T-SQL AVANÇADO..... 10
3.1. Ambiente e Estrutura do Banco de Dados..... 11
3.2. Automação do Banco de Dados com PowerShell.....11
3.3. Organização dos Scripts T-SQL.....12
4. STORED PROCEDURES.....14
4.1. Procedure usp_AssumirChamado..... 14
4.2. Procedure usp_FinalizarChamado.....16
4.3. Parametrização e Integração com a API.....18
5. FUNCTIONS E VIEWS..... 19
5.1. Function ufn_DuracaoMinutos..... 19
5.2. View vw_ChamadosDetalhados.....20
5.3. View vw_ComentariosDetalhados.....21
5.4. Integração das Functions e Views com a API.....23
6. TRIGGERS..... 23
6.1. Trigger trg_Chamados_ValidarDatas..... 24
6.2. Integridade dos Dados e Impacto no Desempenho..... 25
7. TRANSAÇÕES E CONTROLE DE CONCORRÊNCIA..... 26
7.1. Propriedades ACID e Controle Transacional..... 26
7.2. Bloqueios e Níveis de Isolamento..... 27

7.3. Tratamento de Conflitos..... 28
8. CURSORS E FETCH..... 29
8.1. Justificativa da Não Utilização de Cursors..... 30
8.2. Alternativas Baseadas em Conjuntos..... 31
9. OTIMIZAÇÃO DE CONSULTAS E ÍNDICES..... 31
9.1. Índices Implementados..... 32
9.2. Análise dos Planos de Execução..... 34
9.3. Ferramentas de Análise e Monitoramento..... 36
10. SEGURANÇA E BOAS PRÁTICAS..... 36
10.1. Autenticação e Controle de Acesso..... 37
10.2. Proteção Contra SQL Injection..... 38
10.3. Proteção de Senhas e Organização do Código T-SQL..... 38
10.4. Limitações de Segurança Identificadas..... 39
11. COLOCAÇÃO EM PRODUÇÃO..... 40
11.1. Infraestrutura e Configuração do Ambiente..... 40
11.2. Segurança, Backup e Monitoramento..... 42
12. EVOLUÇÃO DO PROJETO..... 43
12.1. Limitações Atuais..... 43
12.2. Propostas de Melhoria..... 44
12.3. Cronograma de Evolução..... 45
13. CONCLUSÃO..... 46
REFERÊNCIAS..... 48

1. INTRODUÇÃO

A utilização de sistemas informatizados para o gerenciamento de serviços de suporte técnico contribui para a organização das solicitações, o acompanhamento dos atendimentos e o controle das atividades realizadas pelas equipes responsáveis. Em ambientes nos quais essas demandas são registradas de maneira descentralizada, podem ocorrer dificuldades na identificação dos responsáveis, no acompanhamento das solicitações e na manutenção do histórico dos atendimentos.

Nesse contexto, o TechHelpSolutions foi desenvolvido como uma aplicação web destinada ao gerenciamento de chamados de suporte técnico. O sistema permite o cadastro de usuários, a abertura e o acompanhamento de chamados, a atribuição de atendimentos a técnicos, a atualização das solicitações e o registro de comentários. Também contempla funcionalidades administrativas e mecanismos de autenticação e autorização, conforme os perfis de acesso definidos.

O projeto integra tecnologias de desenvolvimento frontend, backend e banco de dados, com ênfase na aplicação dos conceitos estudados na disciplina de Banco de Dados Avançado. A solução utiliza o Microsoft SQL Server para o armazenamento das informações e a execução de operações por meio da linguagem Transact-SQL (T-SQL).

Entre os recursos implementados estão Stored Procedures, Functions, Views, Triggers, transações e índices, utilizados para organizar consultas, aplicar regras de negócio, preservar a integridade dos dados e controlar operações concorrentes.

A arquitetura também contempla uma API REST desenvolvida com ASP.NET Core 9 e uma interface web construída com HTML, CSS e JavaScript. A comunicação entre esses componentes permite que os usuários realizem as operações disponibilizadas pelo sistema sem acessar diretamente o banco de dados.

Além disso, foi desenvolvido um script PowerShell para automatizar a criação e a configuração do banco de dados, incluindo a estrutura das tabelas, os objetos T-SQL e os registros iniciais necessários aos testes.

1.1. Objetivo Geral

Desenvolver uma aplicação web integrada ao Microsoft SQL Server para o gerenciamento de chamados de suporte técnico, aplicando conceitos avançados de T-SQL, automação de banco de dados, desenvolvimento de API REST e integração com uma interface frontend funcional.

1.2. Objetivos Específicos

Os objetivos específicos do projeto compreendem:

- Estruturar um banco de dados relacional para armazenar usuários, chamados, categorias, prioridades, situações de atendimento e comentários.
- Automatizar a criação, a configuração e a população inicial do banco de dados por meio de um script PowerShell.
- Implementar Stored Procedures, Functions, Views e Triggers para atender às regras de negócio e aos requisitos de integridade.
- Aplicar transações e mecanismos de controle de concorrência nas operações críticas do sistema.
- Criar índices e analisar planos de execução para avaliar o comportamento das consultas SQL.
- Desenvolver uma API REST integrada ao SQL Server, disponibilizando operações de cadastro, consulta, atualização e exclusão dos principais registros.
- Implementar uma interface web que consuma os endpoints da API e permita a interação dos usuários com as funcionalidades disponíveis.
- Aplicar mecanismos de autenticação, autorização e proteção das informações armazenadas.

1.3. Escopo do Projeto

O escopo do TechHelpSolutions compreende o desenvolvimento de uma solução Full Stack para o gerenciamento de chamados de suporte técnico, integrando uma interface web, uma API REST e um banco de dados SQL Server.

A aplicação contempla três perfis principais de acesso: usuário, técnico e administrador. Os usuários podem registrar e acompanhar suas solicitações; os técnicos podem assumir e finalizar atendimentos; e os administradores possuem acesso a funcionalidades de gerenciamento, conforme as permissões estabelecidas no backend.

O sistema também permite a classificação dos chamados por categoria, prioridade e situação, além do registro de comentários relacionados aos atendimentos.

O projeto foi desenvolvido para fins acadêmicos e de demonstração funcional. Sua implementação contempla os principais recursos necessários ao funcionamento da solução, enquanto aspectos adicionais relacionados à infraestrutura, à segurança operacional e ao monitoramento serão considerados nas seções de colocação em produção e evolução do projeto.

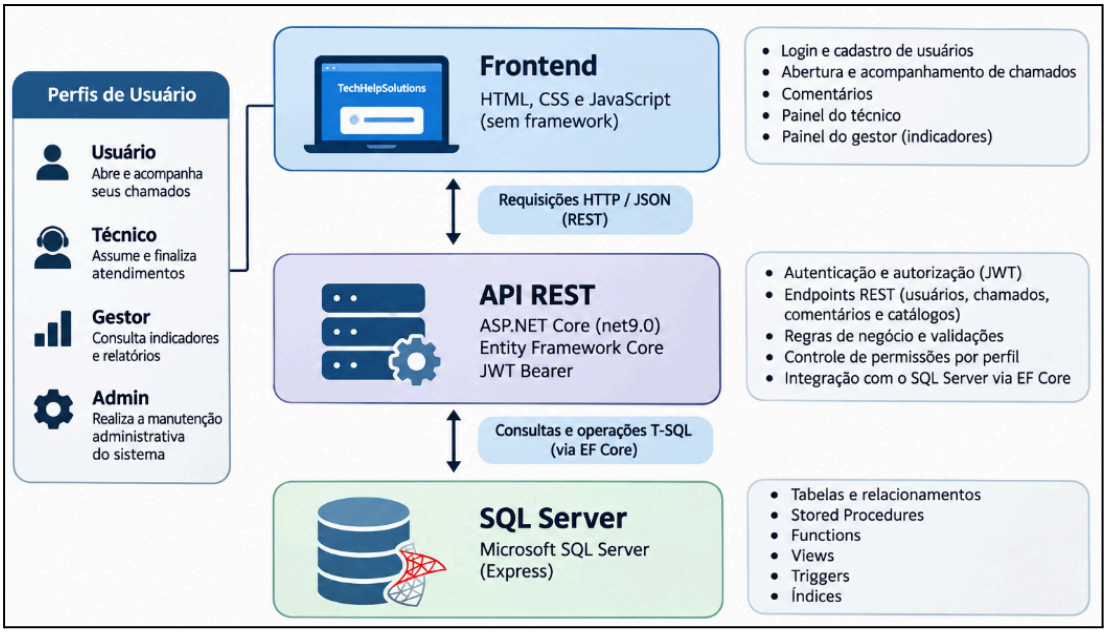
2. ARQUITETURA DO SISTEMA

O TechHelpSolutions foi desenvolvido com uma arquitetura organizada em três camadas principais: apresentação, aplicação e persistência de dados. Essa estrutura permite separar as responsabilidades relacionadas à interação com os usuários, ao processamento das regras de negócio e ao armazenamento das informações.

A comunicação entre os componentes ocorre por meio de uma API REST, responsável por intermediar as requisições enviadas pelo frontend e as operações realizadas no banco de dados SQL Server.

Essa organização contribui para a manutenção do código, a aplicação de mecanismos de segurança e a possibilidade de evolução independente dos componentes do sistema.

Figura 1 - Arquitetura geral do sistema TechHelpSolutions



Fonte: Elaborado pelos autores (2026).

2.1. Camada de Apresentação - Frontend

A camada de apresentação corresponde à interface web do TechHelpSolutions, desenvolvida com HTML, CSS e JavaScript. Sua finalidade é disponibilizar os recursos necessários para que os usuários realizem as operações do sistema de maneira organizada e acessível.

O frontend possui páginas de autenticação, cadastro, abertura de chamados e acompanhamento de atendimentos. As funcionalidades disponibilizadas variam conforme o perfil do usuário autenticado.

O sistema contempla três perfis principais:

- Usuário: pode registrar solicitações de suporte, consultar seus chamados e acompanhar o andamento dos atendimentos.
- Técnico: pode consultar chamados, assumir atendimentos e registrar sua finalização.
- Administrador: possui acesso às funcionalidades administrativas, incluindo operações de gerenciamento e exclusão de chamados autorizadas pela API.

A comunicação com o backend é realizada por meio de requisições HTTP executadas com JavaScript. Os dados enviados pelos formulários são processados pela API, e as respostas recebidas são utilizadas para atualizar as informações apresentadas na interface.

As requisições protegidas incluem um token de autenticação JWT (JSON Web Token), utilizado pelo backend para identificar o usuário e verificar suas permissões de acesso.

2.2. Camada de Aplicação - Backend

A camada de aplicação foi desenvolvida com ASP.NET Core 9, utilizando a linguagem C#. Sua responsabilidade consiste em receber as requisições HTTP, validar os dados fornecidos, aplicar as regras de negócio e executar as operações necessárias no banco de dados.

A API segue os princípios da arquitetura REST e disponibiliza endpoints para autenticação, gerenciamento de usuários, chamados, comentários e consulta aos dados auxiliares do sistema.

O código backend está organizado em componentes com responsabilidades específicas. Os controllers recebem as requisições HTTP, os serviços concentram as operações da aplicação e os Data Transfer Objects (DTOs) definem as estruturas utilizadas na entrada e na saída de informações.

A integração com o SQL Server é realizada por meio do Entity Framework Core 9, responsável pelo mapeamento das entidades e pela execução das operações de persistência.

Além das operações convencionais, o backend utiliza Stored Procedures para executar ações específicas relacionadas à atribuição e à finalização de chamados. Essas operações são processadas diretamente no banco de dados, permitindo a aplicação de verificações e controles transacionais.

A segurança da API utiliza autenticação JWT e autorização baseada em perfis. Dessa forma, o acesso aos endpoints e às operações administrativas é controlado de acordo com as permissões atribuídas a cada usuário.

2.3. Camada de Persistência - Banco de Dados

A camada de persistência utiliza o Microsoft SQL Server Express, responsável pelo armazenamento e pelo gerenciamento das informações do TechHelpSolutions.

O banco de dados possui seis tabelas principais, organizadas conforme as entidades utilizadas pela aplicação:

- Usuarios: armazena informações cadastrais, credenciais protegidas e perfis de acesso.
- Categorias: identifica os tipos de solicitações de suporte.
- Prioridades: define os níveis de prioridade dos atendimentos.
- Status: representa as situações dos chamados.
- Chamados: armazena as solicitações, seus responsáveis, classificações e datas de abertura e fechamento.
- Comentarios: registra as mensagens associadas aos chamados e seus respectivos autores.

Os relacionamentos entre as tabelas são estabelecidos por meio de chaves primárias e estrangeiras, garantindo a integridade referencial dos registros.

Além da estrutura relacional, o banco de dados incorpora recursos avançados de T-SQL, incluindo Stored Procedures, Functions, Views, Triggers, transações e índices.

Esses recursos são utilizados para centralizar determinadas regras de negócio, simplificar consultas, validar informações e apoiar o desempenho das operações realizadas pela aplicação.

2.4. Integração entre as Camadas

A integração entre as camadas ocorre por meio de um fluxo de comunicação que conecta a interface web, a API REST e o banco de dados SQL Server.

Quando um usuário realiza uma operação na interface, como abrir um chamado, o frontend envia uma requisição HTTP ao endpoint correspondente da API.

O backend recebe a solicitação, verifica os dados e as permissões necessárias e executa a operação por meio do Entity Framework Core ou dos objetos T-SQL implementados no banco de dados.

Após o processamento, a API retorna uma resposta HTTP com o resultado da operação. O frontend interpreta essa resposta e apresenta as informações atualizadas ao usuário.

Esse fluxo mantém o acesso ao banco de dados restrito à camada de aplicação, evitando conexões diretas entre a interface web e o SQL Server.

A estrutura do projeto também contempla o arquivo setup_database.ps1, responsável pela inicialização automatizada do banco de dados, e o arquivo README.md, que apresenta as instruções necessárias para instalação, configuração e execução da aplicação.

Dessa forma, a arquitetura adotada integra os componentes necessários ao funcionamento do TechHelpSolutions, mantendo uma separação clara entre apresentação, processamento e persistência das informações.

3. SQL SERVER E T-SQL AVANÇADO

O Microsoft SQL Server foi utilizado como sistema gerenciador de banco de dados relacional do TechHelpSolutions, sendo responsável pelo armazenamento, pela organização e pela integridade das informações utilizadas pela aplicação.

O projeto utiliza a linguagem Transact-SQL (T-SQL) para implementar operações de consulta, manipulação de registros e regras de negócio diretamente no banco de dados. Além das instruções SQL convencionais, foram empregados recursos avançados, como Stored Procedures, Functions, Views, Triggers, transações e índices.

A utilização desses recursos permite organizar as operações de persistência, aplicar validações e apoiar o desempenho das consultas executadas pela API REST.

3.1. Ambiente e Estrutura do Banco de Dados

O ambiente de desenvolvimento utiliza o Microsoft SQL Server 2025 Express, com uma instância local denominada SQLEXPRESS. O banco de dados da aplicação recebe o nome TechHelpSolutions.

A conexão entre a API ASP.NET Core 9 e o SQL Server é realizada por meio do Entity Framework Core 9, utilizando autenticação integrada do Windows no ambiente de desenvolvimento.

A estrutura relacional do banco de dados é composta por seis tabelas principais: Usuarios, Categorias, Prioridades, Status, Chamados e Comentarios.

A tabela Chamados concentra as informações das solicitações de suporte, incluindo título, descrição, usuário solicitante, técnico responsável, categoria, prioridade, situação e datas de abertura e fechamento.

As demais tabelas armazenam os dados necessários à identificação dos usuários, à classificação das solicitações e ao registro das interações realizadas durante os atendimentos.

Os relacionamentos são definidos por chaves primárias e estrangeiras, garantindo a integridade referencial entre os registros. Essa modelagem permite consultar informações relacionadas sem duplicar desnecessariamente os dados armazenados.

Além da estrutura de tabelas, o banco incorpora objetos T-SQL responsáveis por operações específicas do sistema, incluindo atribuição e finalização de chamados, cálculo da duração dos atendimentos e validação de datas.

3.2. Automação do Banco de Dados com PowerShell

Para atender ao requisito de automatização do ambiente, foi desenvolvido o script PowerShell setup_database.ps1, localizado na raiz da pasta src_github.

O script é responsável por preparar o banco de dados, reduzindo a necessidade de executar manualmente os comandos SQL utilizados na configuração da aplicação.

Durante sua execução, são realizadas verificações de acesso à instância SQL Server e das condições necessárias para a inicialização do ambiente.

Quando o banco ainda não está configurado, o processo executa os comandos de criação das tabelas, dos relacionamentos e dos demais componentes estruturais, além de inserir registros iniciais para demonstração e testes.

A carga inicial contempla usuários com diferentes perfis de acesso, categorias, prioridades, situações de atendimento, chamados e comentários.

Após a preparação da estrutura, o script aplica os objetos avançados de T-SQL, respeitando a ordem de execução necessária para atender às dependências entre os componentes.

A automação também contempla verificações da estrutura criada e dos registros esperados, permitindo identificar falhas durante o processo.

Outra característica importante é a possibilidade de reexecutar o script em um ambiente já configurado. Nesse cenário, o procedimento verifica os componentes existentes e evita a duplicação indevida dos registros iniciais.

A preparação automatizada foi validada tanto na instância utilizada durante o desenvolvimento quanto em um ambiente isolado com SQL Server LocalDB, incluindo testes de instalação inicial e reexecução.

Dessa forma, o script PowerShell contribui para a reprodutibilidade do ambiente e facilita a configuração do banco de dados em outras instalações.

3.3. Organização dos Scripts T-SQL

Os scripts SQL do TechHelpSolutions estão organizados no diretório database, localizado dentro da pasta src_github.

O arquivo 01_script.sql contém a estrutura inicial do banco de dados e os registros necessários à sua preparação. Os demais arquivos estão organizados no mesmo diretório, de acordo com suas responsabilidades.

Os principais arquivos utilizados são:

- 01_script.sql: criação da estrutura inicial do banco de dados, incluindo tabelas, relacionamentos e registros necessários aos testes.
- 02_functions.sql: implementação da função ufn_DuracaoMinutos, responsável pelo cálculo da duração dos chamados.
- 03_views.sql: criação das visões vw_ChamadosDetalhados e vw_ComentariosDetalhados, utilizadas para consolidar informações de consultas.
- 04_procedures.sql: implementação das Stored Procedures usp_AssumirChamado e usp_FinalizarChamado, responsáveis por operações críticas do atendimento.
- 05_triggers.sql: criação do Trigger trg_Chamados_ValidarDatas, utilizado para impedir inconsistências entre as datas de abertura e fechamento.
- 06_indexes.sql: definição dos índices adicionais destinados a apoiar consultas frequentes.
- 07_validacao.sql: execução de verificações relacionadas à estrutura e aos objetos implementados no banco de dados.

A separação dos scripts por responsabilidade facilita a identificação dos componentes, a manutenção do código e a execução automatizada das etapas de configuração.

As decisões relacionadas à otimização das consultas consideram a utilização de índices e a análise dos planos de execução do SQL Server. Esses recursos permitem avaliar os métodos de acesso aos dados e identificar possíveis custos associados às consultas.

As características e os resultados dessa análise serão apresentados na seção específica de otimização de consultas e índices.

A organização adotada permite que os objetos avançados de T-SQL sejam mantidos separadamente da estrutura inicial do banco, contribuindo para a clareza e a manutenção da solução.

4. STORED PROCEDURES

As Stored Procedures são procedimentos armazenados no SQL Server que permitem executar conjuntos de instruções T-SQL de maneira organizada e reutilizável. Esses objetos podem receber parâmetros, realizar consultas, modificar registros e controlar transações, contribuindo para a centralização de determinadas regras de negócio no banco de dados.

No TechHelpSolutions, foram implementadas duas Stored Procedures: dbo.usp_AssumirChamado e dbo.usp_FinalizarChamado. Ambas são utilizadas em operações essenciais do gerenciamento de chamados, permitindo controlar a atribuição de técnicos e o encerramento dos atendimentos.

A implementação desses procedimentos contribui para a integridade das informações, pois as verificações necessárias e as atualizações dos registros são executadas dentro de operações transacionais. Além disso, a centralização das instruções SQL facilita a manutenção do código e permite ao SQL Server reutilizar planos de execução quando aplicável.

4.1. Procedure usp_AssumirChamado

A Stored Procedure dbo.usp_AssumirChamado é responsável por atribuir um técnico a um chamado de suporte.

O procedimento recebe os identificadores do chamado e do técnico responsável pela operação. Antes de realizar a atualização, são verificadas as condições necessárias para identificar o registro e confirmar a validade da atribuição.

Quando a operação é autorizada, a tabela Chamados é atualizada com o identificador do técnico e com a situação correspondente ao atendimento em andamento.

A implementação também contempla o controle de alterações concorrentes. Para isso, são utilizados bloqueios de atualização e verificações do estado do registro, evitando que operações simultâneas produzam modificações inconsistentes.

Caso seja identificada uma condição inválida ou um conflito de atualização, o procedimento interrompe a operação e sinaliza o erro, permitindo que a API apresente uma resposta adequada.

A execução ocorre dentro de uma transação, garantindo que as alterações sejam confirmadas somente quando todas as etapas necessárias forem concluídas corretamente.

Figura 2 - Implementação da Stored Procedure usp_AssumirChamado

```
USE [TechHelpSolutions]
GO
/****** Object:  StoredProcedure [dbo].[usp_AssumirChamado]    Script Date: 03/11/2026 19:46:52 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER   PROCEDURE [dbo].[usp_AssumirChamado]
    @ChamadoId int,
    @TecnicoId int,
    @TecnicoIdEsperado int,
    @StatusIdEsperado int,
    @DataFechamentoEsperada datetime2(7)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @TransacoesIniciais int = @@TRANCOUNT;
    BEGIN TRY
        IF @TransacoesIniciais = 0 BEGIN TRANSACTION;
        ELSE SAVE TRANSACTION usp_AssumirChamado;

        DECLARE @IdAtual int, @TecnicoAtual int, @StatusAtual int,
                @FechamentoAtual datetime2(7);
        SELECT @IdAtual = Id, @TecnicoAtual = TecnicoId,
               @StatusAtual = StatusId, @FechamentoAtual = DataFechamento
        FROM dbo.Chamados WITH (UPDLOCK)
        WHERE Id = @ChamadoId;
        IF @IdAtual IS NULL
            THROW 51101, 'Chamado nao encontrado.', 1;

        -- S lock apenas no usuario consultado, retido ate terminar a validacao/escrita.
        IF NOT EXISTS (SELECT 1 FROM dbo.Usuarios WITH (REPEATABLEREAD)
                       WHERE Id = @TecnicoId AND Tipo COLLATE Latin1_General_100_BIN2 = N'Tecnico')
            THROW 51102, 'Tecnico inexistente ou nao elegivel.', 1;

        IF EXISTS (SELECT @TecnicoAtual, @StatusAtual, @FechamentoAtual
                   EXCEPT SELECT @TecnicoIdEsperado, @StatusIdEsperado, @DataFechamentoEsperada)
            THROW 51105, 'O chamado foi alterado por outra operacao. Recarregue e tente novamente.', 1;

        UPDATE dbo.Chamados SET TecnicoId = @TecnicoId, StatusId = 2
        WHERE Id = @ChamadoId;
        -- Assumir preserva DataFechamento, como na NP1. Finalizar usa a hora local da API.
        IF @TransacoesIniciais = 0 COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0
        BEGIN
            -- XACT_ABORT pode inviabilizar tambem a transacao do chamador.
            IF @TransacoesIniciais = 0 OR XACT_STATE() = -1 ROLLBACK TRANSACTION;
            ELSE ROLLBACK TRANSACTION usp_AssumirChamado;
        END;
        THROW;
    END CATCH;
END;
```

Fonte: Elaborado pelos autores (2026).

4.2. Procedure usp_FinalizarChamado

A Stored Procedure dbo.usp_FinalizarChamado é responsável por registrar a conclusão de um atendimento.

O procedimento recebe as informações necessárias para identificar o chamado e o técnico responsável pela finalização. Antes da atualização, verifica a existência do registro, seu estado atual e a correspondência entre o técnico informado e o responsável atribuído ao atendimento.

Essa validação impede que um técnico diferente daquele associado ao chamado realize sua finalização por meio da operação.

Quando as condições são atendidas, o procedimento altera a situação do chamado para finalizado e registra sua data de fechamento.

Assim como ocorre na atribuição, a finalização utiliza controle transacional e mecanismos de concorrência para preservar a consistência dos registros.

Em situações de erro, as alterações não confirmadas são revertidas, evitando que o chamado permaneça com informações parcialmente atualizadas.

Figura 3 - Implementação da Stored Procedure usp_FinalizarChamado

```
USE [TechHelpSolutions]
GO
/****** Object:  StoredProcedure [dbo].[usp_FinalizarChamado]    Script Date: 03/11/2026 19:49:18 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER   PROCEDURE [dbo].[usp_FinalizarChamado]
    @ChamadoId int,
    @TecnicoId int,
    @TecnicoIdEsperado int,
    @StatusIdEsperado int,
    @DataFechamentoEsperada datetime2(7),
    @DataFechamento datetime2(7)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @TransacoesIniciais int = @@TRANCOUNT;
    BEGIN TRY
        IF @TransacoesIniciais = 0 BEGIN TRANSACTION;
        ELSE SAVE TRANSACTION usp_FinalizarChamado;

        DECLARE @IdAtual int, @TecnicoAtual int, @StatusAtual int,
                @FechamentoAtual datetime2(7);
        SELECT @IdAtual = Id, @TecnicoAtual = TecnicoId,
               @StatusAtual = StatusId, @FechamentoAtual = DataFechamento
        FROM dbo.Chamados WITH (UPDLOCK)
        WHERE Id = @ChamadoId;
        IF @IdAtual IS NULL
            THROW 51101, 'Chamado nao encontrado.', 1;

        -- S lock apenas no usuario consultado, retido ate terminar a validacao/escrita.
        IF NOT EXISTS (SELECT 1 FROM dbo.Usuarios WITH (REPEATABLEREAD)
                       WHERE Id = @TecnicoId AND Tipo COLLATE Latin1_General_100_BIN2 = N'Tecnico')
            THROW 51102, 'Tecnico inexistente ou nao elegivel.', 1;

        IF EXISTS (SELECT @TecnicoAtual, @StatusAtual, @FechamentoAtual
                   EXCEPT SELECT @TecnicoIdEsperado, @StatusIdEsperado, @DataFechamentoEsperada)
            THROW 51105, 'O chamado foi alterado por outra operacao. Recarregue e tente novamente.', 1;

        IF @TecnicoAtual IS NULL
            THROW 51103, 'O chamado precisa ser assumido antes de ser finalizado.', 1;
        IF @TecnicoAtual <> @TecnicoId
            THROW 51104, 'Somente o tecnico responsavel pode finalizar o chamado.', 1;
        IF @DataFechamento IS NULL
            THROW 51106, 'Data de fechamento obrigatoria.', 1;

        UPDATE dbo.Chamados SET StatusId = 3, DataFechamento = @DataFechamento
        WHERE Id = @ChamadoId;
        -- Assumir preserva DataFechamento, como na NP1. Finalizar usa a hora local da API.
        IF @TransacoesIniciais = 0 COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0
        BEGIN
            -- XACT_ABORT pode inviabilizar tambem a transacao do chamador.
            IF @TransacoesIniciais = 0 OR XACT_STATE() = -1 ROLLBACK TRANSACTION;
            ELSE ROLLBACK TRANSACTION usp_FinalizarChamado;
        END;
        THROW;
    END CATCH;
END;
```

Fonte: Elaborado pelos autores (2026).

4.3. Parametrização e Integração com a API

As Stored Procedures são executadas pela API REST desenvolvida em ASP.NET Core 9, por meio da camada de serviços responsável pelo gerenciamento dos chamados.

Quando um usuário realiza uma operação de atribuição ou finalização, o frontend envia uma requisição HTTP ao endpoint correspondente. O backend verifica a autenticação, as permissões do usuário e os dados recebidos antes de solicitar a execução do procedimento armazenado.

A comunicação com o SQL Server utiliza comandos parametrizados, permitindo que os identificadores e demais valores necessários sejam enviados separadamente das instruções SQL. Essa abordagem contribui para a prevenção de SQL Injection e para a organização do acesso aos dados.

Os procedimentos utilizam TRY...CATCH, SET XACT_ABORT ON e comandos de controle transacional para tratar falhas durante a execução. A instrução THROW permite sinalizar erros que precisam ser interpretados pela aplicação.

Quando ocorre um conflito relacionado ao estado atual do chamado, a API pode retornar o código HTTP 409 Conflict, indicando que a operação solicitada não pôde ser concluída nas condições encontradas.

A implementação foi submetida a testes de atribuição, finalização, tratamento de erros e concorrência, incluindo cenários com requisições simultâneas sobre o mesmo chamado.

Dessa forma, as Stored Procedures implementadas demonstram a aplicação prática de T-SQL avançado na centralização de regras de negócio, no controle de operações críticas e na integração entre o SQL Server e a API REST do TechHelpSolutions.

5. FUNCTIONS E VIEWS

As Functions e Views são recursos do SQL Server utilizados para organizar e reutilizar operações de consulta e processamento de dados. As Functions permitem encapsular cálculos e retornar valores a partir de parâmetros definidos, enquanto as Views disponibilizam consultas armazenadas que podem ser utilizadas como fontes de dados por outras instruções SQL e aplicações.

No TechHelpSolutions, esses recursos foram implementados para facilitar a obtenção de informações relacionadas aos chamados e comentários, reduzindo a necessidade de repetir consultas complexas na camada de aplicação.

O projeto contempla a função dbo.ufn_DuracaoMinutos e as visões dbo.vw_ChamadosDetalhados e dbo.vw_ComentariosDetalhados, integradas às operações de consulta do sistema.

5.1. Function ufn_DuracaoMinutos

A função dbo.ufn_DuracaoMinutos foi desenvolvida para calcular a duração de um chamado de suporte técnico, considerando a diferença entre sua data de abertura e sua data de fechamento.

A implementação recebe duas datas como parâmetros e retorna o tempo decorrido em minutos. Para realizar o cálculo, utiliza a função nativa DATEDIFF_BIG, considerando a diferença em milissegundos e convertendo o resultado para minutos.

Essa abordagem permite representar durações com precisão decimal, inclusive em atendimentos cuja duração não corresponde a um número inteiro de minutos.

A função também contempla situações específicas que poderiam comprometer a consistência do resultado. Quando a data de fechamento não está preenchida, indicando que o atendimento ainda não foi concluído, o retorno é nulo. Caso a data de fechamento seja anterior à data de abertura, o resultado retornado é zero.

A centralização desse cálculo no banco de dados evita a duplicação da lógica em diferentes consultas e permite que a mesma regra seja utilizada por outros objetos T-SQL.

No TechHelpSolutions, a função é utilizada na composição das informações detalhadas dos chamados, permitindo consultar a duração dos atendimentos finalizados.

Figura 4 - Implementação da Function ufn_DuracaoMinutos

```sql
USE [TechHelpSolutions]
GO
/****** Object:  UserDefinedFunction [dbo].[ufn_DuracaoMinutos]    Script Date: 03/11/2026 20:01:03 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER   FUNCTION [dbo].[ufn_DuracaoMinutos]
(
    @DataAbertura datetime2(7),
    @DataFechamento datetime2(7)
)
RETURNS float
WITH SCHEMABINDING
AS
BEGIN
    RETURN CASE
        WHEN @DataAbertura IS NULL OR @DataFechamento IS NULL THEN NULL
        WHEN @DataFechamento < @DataAbertura THEN CONVERT(float, 0)
        ELSE CONVERT(float, DATEDIFF_BIG(MILLISECOND, @DataAbertura, @DataFechamento)) / 60000.0
    END;
END;
```

Fonte: Elaborado pelos autores (2026).

5.2. View vw_ChamadosDetalhados

A View dbo.vw_ChamadosDetalhados foi implementada para disponibilizar uma consulta consolidada das informações relacionadas aos chamados.

Sua estrutura utiliza relacionamentos entre a tabela Chamados e as tabelas auxiliares do sistema, permitindo recuperar dados que, originalmente, estão distribuídos em diferentes entidades.

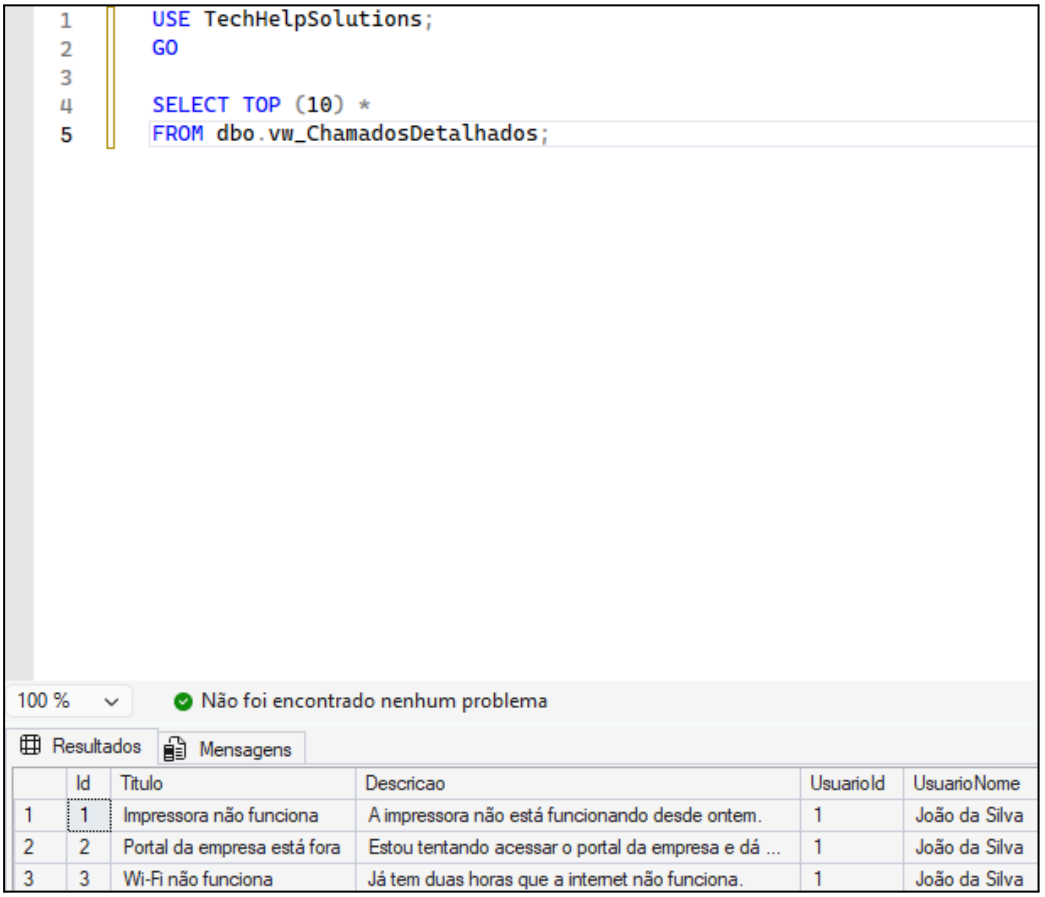
Entre as informações disponibilizadas estão os dados de identificação do chamado, sua descrição, o usuário solicitante, o técnico responsável, a categoria, a prioridade, a situação e as datas relacionadas ao atendimento.

A View também utiliza a função dbo.ufn_DuracaoMinutos para apresentar a duração calculada dos chamados que possuem data de fechamento.

A utilização dessa visão facilita a consulta das informações necessárias à apresentação dos atendimentos, pois concentra os relacionamentos e os campos relevantes em uma única estrutura lógica.

Além disso, a View evita a exposição desnecessária de informações sensíveis armazenadas em outras tabelas, como os dados utilizados na autenticação dos usuários.

Figura 5 - Implementação da View vw_ChamadosDetalhados



	Id	Titulo	Descricao	UsuarioId	UsuarioNome
1	1	Impressora não funciona	A impressora não está funcionando desde ontem.	1	João da Silva
2	2	Portal da empresa está fora	Estou tentando acessar o portal da empresa e dá ...	1	João da Silva
3	3	Wi-Fi não funciona	Já tem duas horas que a internet não funciona.	1	João da Silva

Fonte: Elaborado pelos autores (2026).

5.3. View vw_ComentariosDetalhados

A View dbo.vw_ComentariosDetalhados foi desenvolvida para reunir as informações dos comentários registrados nos chamados de suporte.

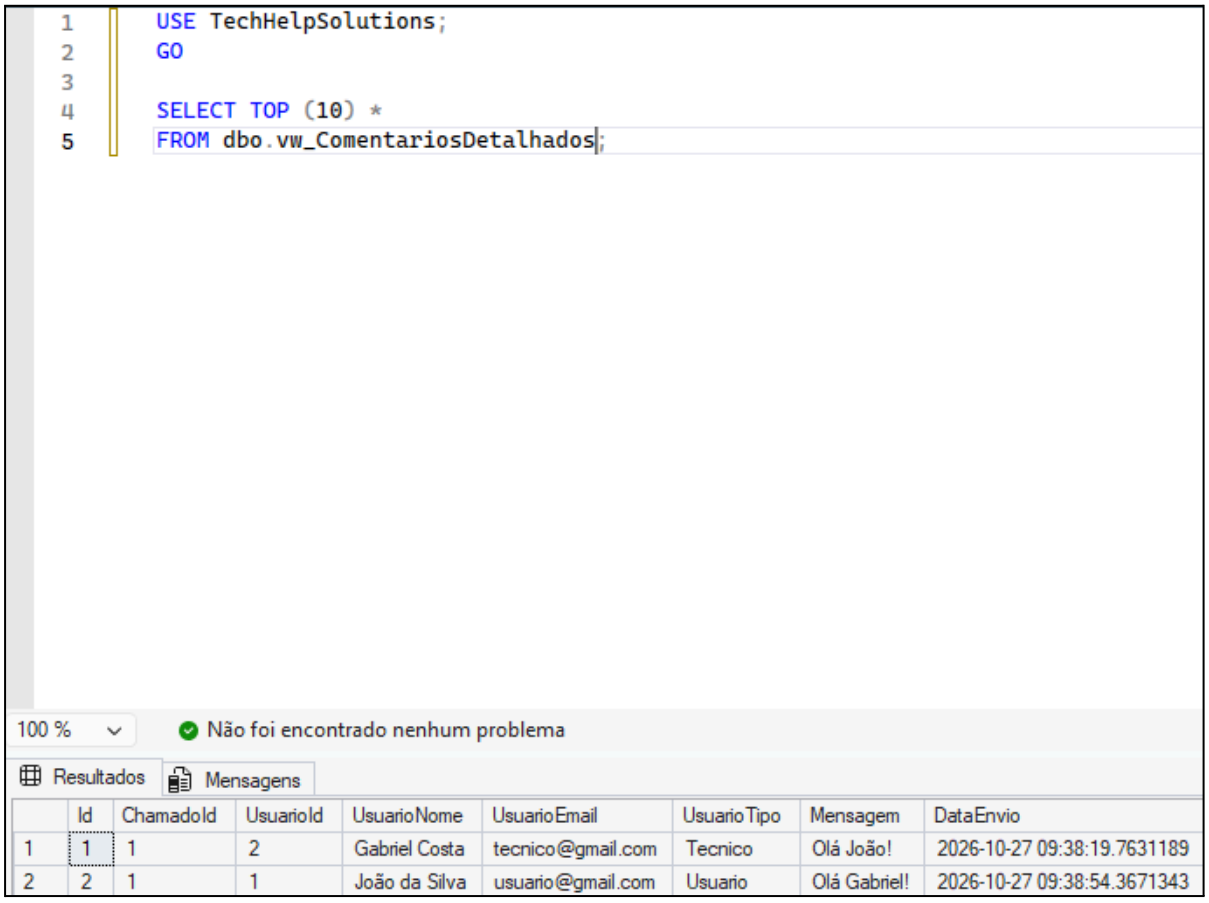
Sua implementação relaciona os registros da tabela Comentarios às informações necessárias para identificar o chamado e o usuário responsável por cada mensagem.

Dessa forma, a consulta permite recuperar o conteúdo dos comentários, seus autores e as respectivas datas de envio, facilitando a apresentação do histórico de interações de um atendimento.

A centralização dessas informações em uma View reduz a repetição de instruções SQL que exigiriam relacionamentos entre diferentes tabelas.

Esse recurso também contribui para a organização das consultas utilizadas pela aplicação, mantendo a lógica de recuperação dos dados detalhados no próprio SQL Server.

Figura 6 - Implementação da View vw_ComentariosDetalhados



	Id	ChamadoId	UsuarioId	UsuarioNome	UsuarioEmail	UsuarioTipo	Mensagem	DataEnvio
1	1	1	2	Gabriel Costa	tecnico@gmail.com	Tecnico	Olá João!	2026-10-27 09:38:19.7631189
2	2	1	1	João da Silva	usuario@gmail.com	Usuario	Olá Gabriel!	2026-10-27 09:38:54.3671343

Fonte: Elaborado pelos autores (2026).

5.4. Integração das Functions e Views com a API

As Views implementadas são utilizadas pela API por meio do Entity Framework Core 9, que permite mapear os resultados das consultas para estruturas utilizadas na aplicação.

Como as Views não representam tabelas convencionais destinadas à inserção e à atualização de registros, seus resultados são configurados no Entity Framework Core como entidades sem chave primária (*keyless entities*), destinadas principalmente às operações de leitura.

Essa configuração permite consultar as informações consolidadas diretamente pelo backend, sem a necessidade de reconstruir todos os relacionamentos em cada operação.

A utilização conjunta de Functions e Views demonstra a aplicação de recursos avançados de T-SQL para organizar consultas, reutilizar cálculos e simplificar o acesso às informações.

No TechHelpSolutions, esses objetos complementam as Stored Procedures e contribuem para a separação entre as operações de consulta e as regras de negócio responsáveis pela alteração dos registros.

6. TRIGGERS

Os Triggers são objetos do SQL Server que permitem executar automaticamente instruções T-SQL em resposta a determinadas operações realizadas no banco de dados, como inserções, atualizações e exclusões de registros.

Esse recurso pode ser utilizado para implementar validações, reforçar regras de integridade e executar ações relacionadas às modificações dos dados. Entretanto, sua utilização exige atenção, pois os comandos são executados automaticamente e podem aumentar o custo das operações.

No TechHelpSolutions, foi implementado o Trigger dbo.trg_Chamados_ValidarDatas, responsável por verificar a consistência das datas de abertura e fechamento dos chamados.

6.1. Trigger trg_Chamados_ValidarDatas

O Trigger dbo.trg_Chamados_ValidarDatas foi desenvolvido para impedir o armazenamento de chamados cuja data de fechamento seja anterior à data de abertura.

Essa validação é necessária porque a data de fechamento representa a conclusão de um atendimento e, portanto, não deve anteceder o momento em que a solicitação foi registrada.

O Trigger foi definido como AFTER INSERT, UPDATE na tabela Chamados. Dessa forma, sua execução ocorre automaticamente quando novos registros são inseridos ou quando informações existentes são atualizadas.

Durante a execução, o Trigger consulta a tabela lógica inserted, disponibilizada pelo SQL Server para representar os registros envolvidos na operação.

A validação verifica se existem chamados com data de fechamento preenchida e anterior à respectiva data de abertura.

Quando essa condição é identificada, o Trigger utiliza a instrução THROW para sinalizar o erro e impedir a confirmação da operação inválida.

A implementação considera o processamento de múltiplos registros em uma única instrução SQL, utilizando uma abordagem baseada em conjuntos, sem a necessidade de percorrer individualmente cada linha modificada.

Essa característica permite que a regra seja aplicada tanto em operações individuais quanto em atualizações que envolvam vários chamados simultaneamente.

Figura 7 - Implementação do Trigger trg_Chamados_ValidarDatas

```sql
USE [TechHelpSolutions]
GO
/****** Object:  Trigger [dbo].[trg_Chamados_ValidarDatas]    Script Date: 03/11/2026 20:09:26 ******/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER   TRIGGER [dbo].[trg_Chamados_ValidarDatas]
ON [dbo].[Chamados]
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    -- Um unico registro invalido rejeita toda a operacao, inclusive em lotes.
    -- NULL e igualdade sao permitidos. Nao corrige nem sobrescreve valores.
    IF EXISTS (
        SELECT 1 FROM inserted
        WHERE DataFechamento IS NOT NULL AND DataFechamento < DataAbertura
    )
        THROW 51201, 'A data de fechamento nao pode ser anterior a data de abertura.', 1;
END;
```

Fonte: Elaborado pelos autores (2026).

6.2. Integridade dos Dados e Impacto no Desempenho

A utilização do Trigger contribui para a integridade dos dados porque estabelece uma validação diretamente no banco de dados, independentemente da origem da operação.

Dessa forma, mesmo que uma atualização seja executada fora da API, a regra continuará sendo aplicada pelo SQL Server.

Essa proteção complementa as validações realizadas na camada de aplicação e reduz a possibilidade de armazenamento de informações inconsistentes.

Entretanto, os Triggers também apresentam impactos que devem ser considerados. Como sua execução ocorre dentro da transação associada à operação que os acionou, instruções complexas ou consultas excessivas podem aumentar o tempo de processamento e a duração dos bloqueios sobre os registros.

Por esse motivo, o Trigger implementado no TechHelpSolutions possui uma responsabilidade específica e utiliza uma consulta objetiva para identificar inconsistências nas datas.

A integração com o Entity Framework Core também exigiu uma configuração específica para as operações de gravação na tabela Chamados, utilizando UseSqlOutputClause(false). Essa configuração evita incompatibilidades entre a cláusula OUTPUT utilizada em determinadas operações do SQL Server e a existência de um Trigger habilitado na tabela.

A implementação foi submetida a testes envolvendo inserções e atualizações com datas válidas e inválidas, incluindo operações que modificam múltiplos registros.

Assim, o Trigger desenvolvido demonstra a aplicação prática de validações automáticas no SQL Server, reforçando a consistência das informações sem transferir toda a responsabilidade pela integridade dos dados para a API REST.

7. TRANSAÇÕES E CONTROLE DE CONCORRÊNCIA

As transações são mecanismos utilizados pelos sistemas gerenciadores de banco de dados para garantir a execução consistente de operações que envolvem uma ou mais alterações nos registros. No SQL Server, permitem agrupar instruções T-SQL em uma unidade de trabalho, assegurando que as modificações sejam confirmadas ou revertidas conforme o resultado da execução.

O controle de concorrência, por sua vez, é necessário quando diferentes usuários ou processos acessam e modificam os mesmos registros simultaneamente. Sem mecanismos adequados, essas operações podem produzir conflitos, perda de atualizações e inconsistências nas informações armazenadas.

No TechHelpSolutions, os recursos transacionais foram aplicados principalmente às Stored Procedures responsáveis pela atribuição e pela finalização de chamados, garantindo maior segurança nas operações críticas do atendimento.

7.1. Propriedades ACID e Controle Transacional

O SQL Server utiliza as propriedades ACID para assegurar a confiabilidade das transações realizadas no banco de dados.

Essas propriedades compreendem:

- Atomicidade: garante que as operações de uma transação sejam executadas integralmente ou revertidas em caso de falha.
- Consistência: assegura que as alterações respeitem as regras de integridade e as restrições definidas no banco de dados.
- Isolamento: controla a interação entre transações simultâneas, reduzindo interferências indevidas durante o processamento.
- Durabilidade: garante que as alterações confirmadas permaneçam registradas, mesmo após falhas posteriores do sistema.

No TechHelpSolutions, as Stored Procedures dbo.usp_AssumirChamado e dbo.usp_FinalizarChamado utilizam transações explícitas para controlar as modificações realizadas na tabela Chamados.

A execução é iniciada por meio de BEGIN TRANSACTION, enquanto a confirmação das alterações ocorre com COMMIT TRANSACTION, após a conclusão das verificações e atualizações necessárias.

Em situações de erro, a instrução ROLLBACK TRANSACTION permite desfazer as alterações ainda não confirmadas, preservando o estado consistente dos registros.

Os procedimentos também utilizam TRY...CATCH para organizar o tratamento de exceções e SET XACT_ABORT ON para determinar o encerramento da transação diante de erros de execução que exigem sua reversão.

A instrução THROW é utilizada para sinalizar condições inválidas, permitindo que a aplicação identifique a falha e apresente uma resposta apropriada.

Dessa forma, operações como a atribuição de um técnico e a finalização de um atendimento são executadas de maneira controlada, evitando modificações parciais nos dados.

7.2. Bloqueios e Níveis de Isolamento

O controle de concorrência do SQL Server utiliza mecanismos de bloqueio para coordenar o acesso aos registros durante a execução de transações simultâneas.

Os níveis de isolamento determinam quais alterações realizadas por outras transações podem ser observadas durante uma operação. Entre os níveis disponibilizados pelo SQL Server estão READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SNAPSHOT e SERIALIZABLE.

Cada nível apresenta características específicas relacionadas à consistência das leituras, à possibilidade de interferência entre transações e ao custo de processamento.

No TechHelpSolutions, o controle de concorrência das operações críticas utiliza o bloqueio UPDLOCK, aplicado durante a consulta dos registros que poderão ser modificados.

Esse recurso solicita um bloqueio de atualização sobre os dados consultados, contribuindo para coordenar transações que pretendem alterar o mesmo chamado.

A utilização de UPDLOCK é especialmente importante na atribuição de atendimentos, pois diferentes técnicos podem tentar assumir uma mesma solicitação em momentos próximos.

Ao combinar o bloqueio com verificações do estado atual do chamado, o procedimento reduz o risco de alterações conflitantes e permite que as operações sejam processadas de maneira consistente.

A implementação utiliza os mecanismos de isolamento e bloqueio do SQL Server sem depender da criação de um sistema próprio de controle de concorrência na aplicação.

7.3. Tratamento de Conflitos

O tratamento de conflitos é necessário quando uma operação não pode ser concluída porque as condições do registro foram alteradas ou não atendem às regras estabelecidas.

No TechHelpSolutions, esse controle está presente nas Stored Procedures de atribuição e finalização de chamados.

Durante a atribuição, o procedimento consulta o registro e verifica as condições necessárias antes de atualizar o técnico responsável e a situação do atendimento.

Na finalização, são verificadas as informações do chamado e a correspondência entre o técnico responsável e aquele que solicita o encerramento.

Quando uma operação não atende às condições exigidas, o procedimento sinaliza o erro e impede a confirmação de alterações indevidas.

A API interpreta os conflitos identificados e pode retornar o código HTTP 409 Conflict, informando que a solicitação não pôde ser executada devido ao estado atual do recurso.

Foram realizados testes envolvendo requisições simultâneas para verificar o comportamento das operações de atribuição e finalização sobre os mesmos registros.

Esses testes permitem avaliar a atuação dos bloqueios, das validações e do controle transacional em situações nas quais diferentes operações competem pelo acesso a um chamado.

Assim, a utilização de transações, bloqueios e tratamento de erros contribui para preservar a integridade dos dados e garantir maior confiabilidade às operações críticas do TechHelpSolutions.

8. CURSORS E FETCH

Os Cursors são recursos do SQL Server que permitem percorrer individualmente os registros retornados por uma consulta. Sua utilização envolve a declaração de um cursor, a abertura do conjunto de resultados e a recuperação sequencial das linhas por meio da instrução FETCH.

Esse mecanismo pode ser empregado em situações que exigem processamento individualizado dos registros, especialmente quando uma operação depende de informações obtidas em etapas anteriores.

Entretanto, o processamento linha a linha pode apresentar maior custo computacional em comparação às operações baseadas em conjuntos, principalmente quando envolve grandes volumes de dados.

No TechHelpSolutions, optou-se por não utilizar Cursors nas operações implementadas, priorizando instruções T-SQL capazes de processar múltiplos registros de maneira conjunta.

8.1. Justificativa da Não Utilização de Cursors

A decisão de não utilizar Cursors no TechHelpSolutions está relacionada às características das operações realizadas pelo sistema.

As principais funcionalidades envolvem consultas, inserções, atualizações e exclusões de registros, que podem ser executadas diretamente por instruções SQL convencionais.

As Stored Procedures responsáveis pela atribuição e pela finalização de chamados realizam operações direcionadas aos registros identificados pelos parâmetros recebidos, sem necessidade de percorrer sequencialmente um conjunto de resultados.

Da mesma forma, as Views implementadas utilizam consultas relacionais para consolidar informações, enquanto a Function de duração realiza um cálculo a partir das datas fornecidas.

O Trigger de validação de datas também dispensa a utilização de Cursors, pois verifica os registros afetados por uma operação utilizando uma consulta baseada em conjuntos.

Dessa forma, a utilização de processamento linha a linha não apresentaria benefícios para os requisitos implementados e poderia introduzir complexidade desnecessária ao código T-SQL.

8.2. Alternativas Baseadas em Conjuntos

O processamento baseado em conjuntos (*set-based processing*) permite que o SQL Server execute operações sobre grupos de registros por meio de instruções como SELECT, INSERT, UPDATE e DELETE.

Essa abordagem aproveita os mecanismos de otimização do banco de dados, permitindo que o SQL Server determine estratégias de execução adequadas às consultas realizadas.

No TechHelpSolutions, essa característica está presente principalmente nas Views e no Trigger de validação de datas.

As Views utilizam operações JOIN para relacionar informações armazenadas em diferentes tabelas, disponibilizando resultados consolidados sem a necessidade de percorrer individualmente os registros.

O Trigger dbo.trg_Chamados_ValidarDatas, por sua vez, consulta a tabela lógica inserted para identificar possíveis inconsistências em todos os registros afetados por uma instrução de inserção ou atualização.

Essa implementação é importante porque um Trigger pode ser acionado por comandos que modificam múltiplas linhas simultaneamente. Ao utilizar uma verificação baseada em conjuntos, a validação permanece aplicável independentemente da quantidade de registros envolvidos.

Portanto, a ausência de Cursors no TechHelpSolutions representa uma decisão técnica fundamentada nas necessidades da aplicação. As operações implementadas são atendidas por instruções relacionais e mecanismos transacionais, mantendo o código mais simples e evitando o custo adicional de processamento sequencial desnecessário.

9. OTIMIZAÇÃO DE CONSULTAS E ÍNDICES

A otimização de consultas é uma etapa importante no desenvolvimento de sistemas que utilizam bancos de dados relacionais, pois permite avaliar a eficiência das operações de recuperação e manipulação das informações.

No Microsoft SQL Server, o desempenho das consultas pode ser influenciado por fatores como a estrutura das tabelas, os relacionamentos, a quantidade de registros, os índices disponíveis e as estratégias selecionadas pelo otimizador de consultas.

No TechHelpSolutions, foram implementados índices adicionais para apoiar operações frequentes de consulta de chamados e comentários. Também foram realizadas análises dos planos de execução e das estatísticas de leitura, permitindo observar o comportamento das consultas antes e depois da criação desses índices.

9.1. Índices Implementados

Os índices são estruturas utilizadas pelo SQL Server para facilitar a localização de registros, reduzindo, em determinadas situações, a necessidade de percorrer integralmente as tabelas.

A escolha dos índices deve considerar os campos utilizados em filtros, relacionamentos e ordenações, além do impacto sobre as operações de inserção, atualização e exclusão.

No TechHelpSolutions, foram criados três índices adicionais:

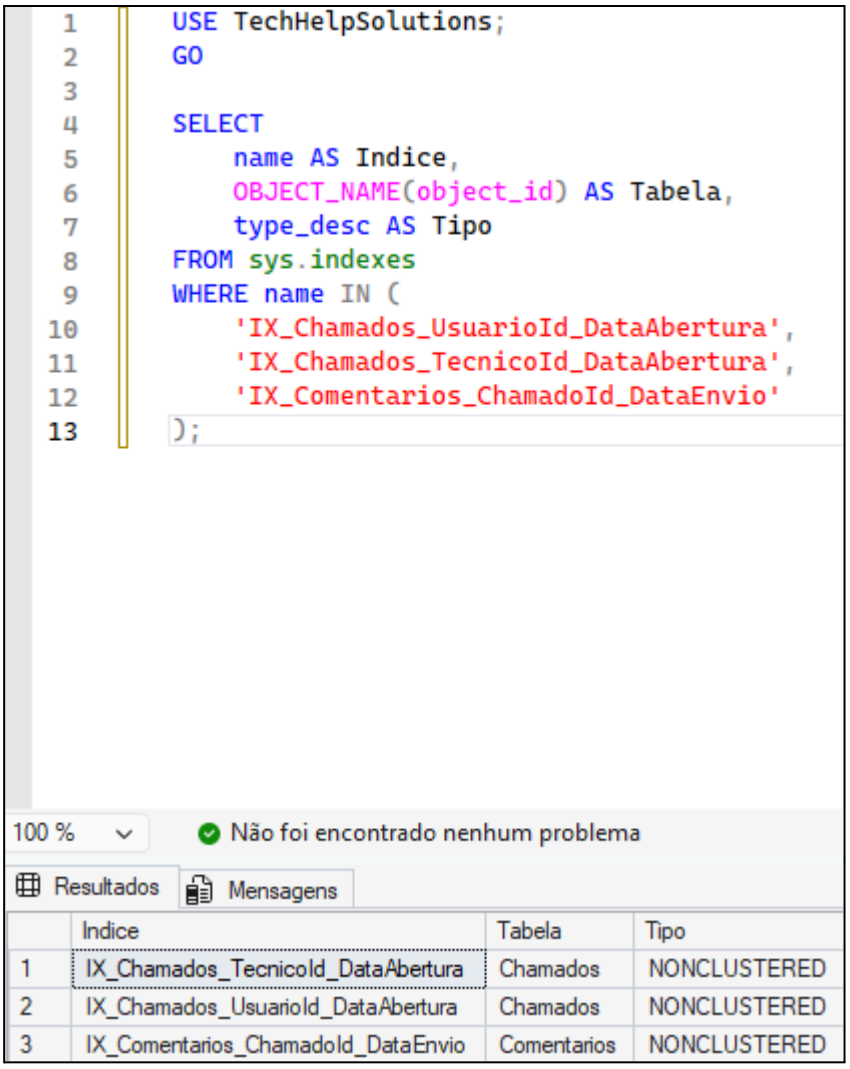
- IX_Chamados_UsuarioId_DataAbertura: criado na tabela Chamados, utilizando os campos UsuarioId e DataAbertura. Seu objetivo é apoiar consultas que recuperam os chamados de determinado usuário, especialmente quando os resultados são organizados pela data de abertura.
- IX_Chamados_TecnicoId_DataAbertura: criado na tabela Chamados, utilizando os campos TecnicoId e DataAbertura. Sua finalidade é apoiar a recuperação dos atendimentos associados a um técnico.
- IX_Comentarios_ChamadoId_DataEnvio: criado na tabela Comentarios, utilizando os campos ChamadoId e DataEnvio. Esse índice foi definido para auxiliar consultas que recuperam o histórico de comentários de um chamado em ordem cronológica.

A seleção desses campos está relacionada às funcionalidades disponibilizadas pela aplicação, como o acompanhamento de solicitações, a consulta dos atendimentos atribuídos aos técnicos e a visualização dos comentários registrados.

Considerando os índices adicionais e aqueles associados às estruturas já existentes, como chaves primárias e estrangeiras, o banco de dados apresenta um total de 17 índices.

A implementação foi organizada no arquivo 06_indexes.sql, permitindo que os índices sejam criados durante a configuração automatizada do banco de dados.

Figura 8 - Implementação dos Índices



Fonte: Elaborado pelos autores (2026).

9.2. Análise dos Planos de Execução

Os planos de execução permitem visualizar as operações utilizadas pelo SQL Server para processar uma consulta, incluindo os métodos de acesso às tabelas e aos índices.

Entre as operações que podem aparecer nesses planos estão Table Scan, Index Scan e Index Seek. A escolha depende das características da consulta, das estatísticas disponíveis e da estimativa de custo calculada pelo otimizador.

No TechHelpSolutions, foram realizadas comparações antes e depois da criação dos índices adicionais, considerando consultas relacionadas aos usuários, técnicos e comentários.

As análises identificaram alterações nos métodos de acesso utilizados pelo SQL Server, incluindo a utilização de operações Index Seek após a criação dos índices.

Entretanto, os resultados das estatísticas de leitura lógica demonstraram que a criação dos índices não produziu redução imediata no número de páginas lidas durante os testes realizados.

Os resultados observados foram:

Consulta analisada	Antes dos índices	Depois dos índices
Chamados por usuário	32 leituras lógicas	37 leituras lógicas
Chamados por técnico	23 leituras lógicas	26 leituras lógicas
Comentários por chamado	6 leituras lógicas	10 leituras lógicas

Os valores demonstram um pequeno aumento nas leituras lógicas após a implementação dos índices.

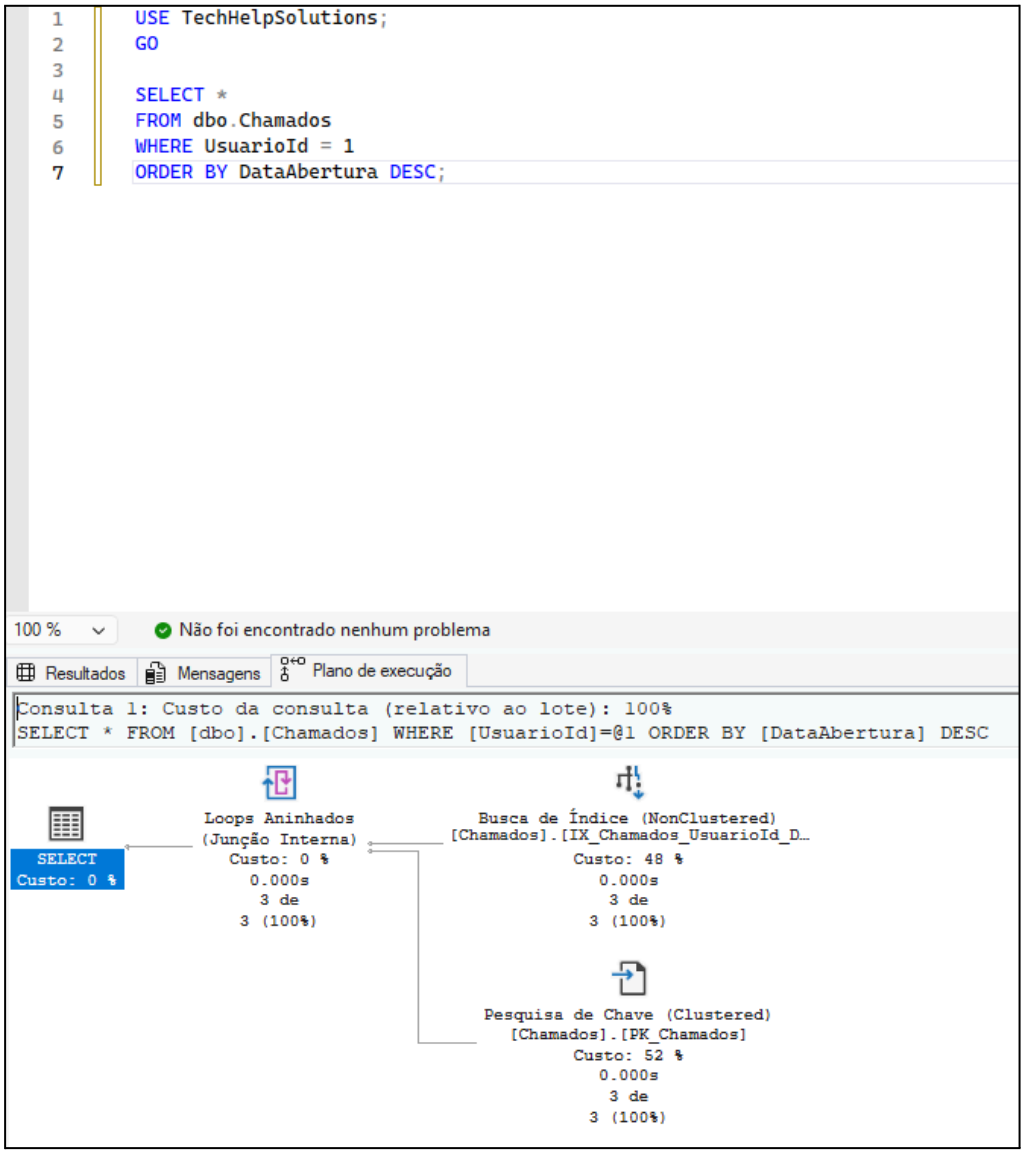
Esse comportamento deve ser considerado em conjunto com o volume reduzido de registros utilizado nos testes. Em bases pequenas, a leitura direta de uma tabela pode apresentar custo semelhante ou inferior ao acesso por índices, especialmente quando a consulta exige a recuperação de colunas adicionais.

Portanto, os resultados não permitem afirmar que os índices produziram ganhos de desempenho no ambiente testado.

Apesar disso, os índices foram mantidos por estarem alinhados aos padrões de consulta da aplicação e por apresentarem potencial de utilização em cenários com maior volume de dados.

A avaliação de sua efetividade deverá ser repetida com uma quantidade mais representativa de registros, considerando também o tempo de execução, o consumo de recursos e o custo de manutenção dos índices.

Figura 9 - Plano de execução de consulta de chamados por usuário



Fonte: Elaborado pelos autores (2026).

9.3. Ferramentas de Análise e Monitoramento

O SQL Server disponibiliza ferramentas e recursos que permitem analisar o comportamento das consultas e identificar possíveis oportunidades de otimização.

No TechHelpSolutions, a avaliação foi realizada com o auxílio dos planos de execução e das estatísticas de leitura lógica, permitindo comparar as operações utilizadas pelo banco de dados.

O comando SET STATISTICS IO ON permite identificar a quantidade de leituras realizadas durante a execução de consultas. Já o recurso de plano de execução do SQL Server Management Studio (SSMS) possibilita visualizar os operadores selecionados pelo otimizador.

Além desses recursos, o SQL Server disponibiliza ferramentas como o Query Store e as Dynamic Management Views (DMVs), que podem ser utilizadas em análises futuras para acompanhar consultas recorrentes, planos de execução e informações de desempenho.

A utilização contínua desses mecanismos é importante porque o comportamento das consultas pode mudar conforme o crescimento das tabelas, a distribuição dos registros e a frequência das operações.

Assim, a otimização implementada no TechHelpSolutions não se limita à criação de índices, mas contempla a análise de evidências de execução e o reconhecimento de que a eficiência de uma estrutura deve ser verificada por meio de medições.

Os testes realizados demonstram a importância de avaliar os resultados obtidos antes de concluir que uma alteração no banco de dados representa uma melhoria efetiva de desempenho.

10. SEGURANÇA E BOAS PRÁTICAS

A segurança de uma aplicação integrada a um banco de dados envolve a proteção das informações armazenadas, o controle das operações permitidas e a prevenção de acessos não autorizados.

No TechHelpSolutions, foram implementados mecanismos de autenticação, autorização, proteção de senhas e parametrização de comandos SQL. Esses recursos contribuem para reduzir vulnerabilidades e estabelecer controles sobre as funcionalidades disponibilizadas pela API REST.

Além das medidas implementadas, foram identificados aspectos que exigem ajustes antes da utilização do sistema em um ambiente de produção.

10.1. Autenticação e Controle de Acesso

O TechHelpSolutions utiliza autenticação baseada em JSON Web Token (JWT), implementada no backend desenvolvido com ASP.NET Core 9.

Durante o processo de autenticação, o usuário informa suas credenciais, que são verificadas pela API. Quando a autenticação é realizada com sucesso, o sistema emite um token contendo informações necessárias à identificação do usuário e ao controle de suas permissões.

Esse token é enviado nas requisições HTTP destinadas aos endpoints protegidos, permitindo que o backend valide a identidade do solicitante antes de executar as operações.

O sistema contempla três perfis de acesso: usuário, técnico e administrador. As permissões são definidas conforme as responsabilidades de cada perfil, restringindo determinadas funcionalidades aos usuários autorizados.

As verificações de autorização são realizadas no backend, evitando que a segurança dependa exclusivamente da exibição ou ocultação de funcionalidades na interface web.

Essa separação é importante porque requisições à API podem ser realizadas independentemente do frontend. Dessa forma, a validação das permissões deve ocorrer antes da execução das operações protegidas.

10.2. Proteção Contra SQL Injection

SQL Injection é uma vulnerabilidade que ocorre quando dados fornecidos por usuários são incorporados indevidamente à estrutura de comandos SQL, permitindo a alteração do comportamento esperado das consultas.

Para reduzir esse risco, o TechHelpSolutions utiliza os mecanismos de parametrização disponibilizados pelo Entity Framework Core e pela integração com os procedimentos armazenados do SQL Server.

Nas operações convencionais de persistência, o Entity Framework Core realiza o mapeamento das entidades e o envio dos valores utilizados nas consultas.

Nas operações que executam Stored Procedures, os parâmetros são transmitidos separadamente das instruções SQL, evitando a construção de comandos por concatenação direta de informações recebidas da interface.

Essa abordagem permite que os valores sejam tratados como dados, e não como partes executáveis da instrução SQL.

A proteção também é complementada pelas validações realizadas na API, responsáveis por verificar informações recebidas antes do processamento das operações.

10.3. Proteção de Senhas e Organização do Código T-SQL

O armazenamento seguro de senhas é um requisito importante para impedir a exposição direta das credenciais dos usuários em caso de acesso indevido ao banco de dados.

No TechHelpSolutions, as senhas são protegidas por meio do algoritmo PBKDF2-HMAC-SHA256, utilizando um salt aleatório de 16 bytes, uma chave derivada de 32 bytes e 600.000 iterações.

Esse mecanismo permite armazenar uma representação derivada da senha, em vez de manter seu conteúdo original em texto simples.

Durante a autenticação, a senha informada é processada com os parâmetros correspondentes ao registro armazenado. O resultado é comparado por meio de uma operação de tempo constante, reduzindo riscos associados a diferenças de tempo na verificação.

A preparação automatizada do banco de dados também contempla o tratamento das credenciais dos usuários utilizados nos testes, garantindo que os registros iniciais sejam armazenados com a proteção definida pela aplicação.

Em relação ao código T-SQL, os objetos foram organizados em arquivos específicos para Functions, Views, Stored Procedures, Triggers e índices.

Essa separação facilita a manutenção, a identificação de responsabilidades e a execução automatizada dos scripts.

As Stored Procedures utilizam tratamento estruturado de erros e controle transacional, enquanto os Triggers e as consultas priorizam operações baseadas em conjuntos.

Essas práticas contribuem para a legibilidade do código e para a consistência das operações realizadas no SQL Server.

10.4. Limitações de Segurança Identificadas

Apesar dos mecanismos implementados, o ambiente atual apresenta configurações adequadas ao desenvolvimento e à demonstração acadêmica que precisam ser revistas antes de uma implantação em produção.

Entre os principais pontos identificados estão o armazenamento da chave de assinatura JWT no código-fonte, a configuração permissiva de CORS e a utilização de HTTP no ambiente de desenvolvimento.

Além disso, o frontend utiliza o armazenamento local do navegador para manter o token de autenticação, o que exige atenção especial à prevenção de vulnerabilidades como Cross-Site Scripting (XSS).

Outro aspecto identificado é que a alteração de senha não provoca automaticamente a revogação dos tokens JWT já emitidos, permitindo que permaneçam válidos até sua expiração.

Para um ambiente de produção, recomenda-se transferir os segredos de autenticação para um mecanismo seguro de configuração, exigir conexões HTTPS, restringir as origens autorizadas pelo CORS e avaliar estratégias mais seguras de gerenciamento e revogação de sessões.

Também devem ser revisadas as credenciais utilizadas para demonstração e aplicados controles de acesso específicos à conta responsável pela conexão da aplicação com o SQL Server, seguindo o princípio do menor privilégio.

Dessa forma, o TechHelpSolutions apresenta mecanismos fundamentais de segurança implementados, ao mesmo tempo em que reconhece as limitações do ambiente acadêmico e as adequações necessárias para uma operação segura em produção.

11. COLOCAÇÃO EM PRODUÇÃO

A colocação em produção corresponde ao processo de disponibilização de um sistema em um ambiente destinado à utilização contínua pelos usuários. Essa etapa envolve a preparação da infraestrutura, a configuração dos serviços, a aplicação de mecanismos de segurança e a definição de procedimentos para monitoramento e recuperação de dados.

O TechHelpSolutions foi desenvolvido e validado em ambiente local, utilizando tecnologias que permitem sua implantação em uma infraestrutura de servidores. Entretanto, a disponibilização em produção exige configurações adicionais para garantir a disponibilidade, a segurança e a confiabilidade da aplicação.

11.1. Infraestrutura e Configuração do Ambiente

A infraestrutura necessária para a implantação do TechHelpSolutions deve contemplar os três componentes principais da arquitetura: frontend, backend e banco de dados.

O frontend, desenvolvido com HTML, CSS e JavaScript, pode ser disponibilizado por meio de um servidor web responsável pela entrega dos arquivos estáticos aos navegadores dos usuários.

O backend, desenvolvido com ASP.NET Core 9, necessita de um ambiente compatível com a execução de aplicações .NET e de um serviço responsável por receber e encaminhar as requisições HTTP.

O banco de dados utiliza o Microsoft SQL Server, responsável pelo armazenamento das informações e pela execução dos objetos T-SQL implementados.

Para uma implantação inicial, esses componentes podem ser configurados em uma infraestrutura de pequeno porte, desde que sejam considerados os requisitos de processamento, memória, armazenamento e quantidade de acessos simultâneos.

A comunicação entre a API e o SQL Server deve ocorrer por meio de uma conexão configurada especificamente para o ambiente de produção, evitando a utilização de credenciais administrativas ou permissões superiores às necessárias.

O projeto disponibiliza o script setup_database.ps1, que automatiza a preparação da estrutura do banco de dados e a criação dos objetos utilizados pela aplicação.

Esse recurso facilita a configuração inicial do ambiente, mas sua execução em produção deve ser precedida da revisão dos dados de demonstração e das configurações utilizadas durante o desenvolvimento.

Também é necessário definir variáveis de ambiente ou mecanismos seguros de configuração para armazenar informações sensíveis, como credenciais de conexão e chaves de autenticação.

A disponibilização da aplicação deve utilizar HTTPS, garantindo a criptografia das informações transmitidas entre os navegadores e o servidor.

Além disso, recomenda-se restringir o acesso direto ao SQL Server, permitindo conexões apenas dos componentes autorizados e das ferramentas administrativas necessárias.

11.2. Segurança, Backup e Monitoramento

A manutenção de um sistema em produção exige procedimentos contínuos para proteger as informações e identificar falhas que possam comprometer seu funcionamento.

No TechHelpSolutions, as medidas de segurança implementadas durante o desenvolvimento devem ser complementadas por configurações específicas do ambiente de implantação.

Entre essas medidas estão a utilização de conexões criptografadas, a restrição das origens permitidas pelo CORS, a proteção das chaves JWT e a aplicação do princípio do menor privilégio às contas de acesso ao banco de dados.

Também devem ser estabelecidos procedimentos de backup para permitir a recuperação das informações em situações de falha, exclusão acidental ou comprometimento do ambiente.

Uma estratégia adequada deve contemplar cópias periódicas do banco de dados, armazenamento protegido dos arquivos gerados e testes de restauração para verificar a integridade dos backups.

A frequência das cópias deve ser definida conforme a importância das informações e a quantidade de alterações realizadas no sistema.

O monitoramento da aplicação deve considerar tanto o funcionamento da API quanto o comportamento do SQL Server.

No backend, recomenda-se acompanhar erros de execução, falhas de autenticação, disponibilidade dos endpoints e tempo de resposta das requisições.

No banco de dados, devem ser observados indicadores como utilização de recursos, consultas com elevado custo de execução, bloqueios prolongados e crescimento das tabelas.

Ferramentas como SQL Server Management Studio, Query Store e Dynamic Management Views podem auxiliar na identificação de problemas de desempenho e na análise das operações realizadas.

Além disso, a utilização de registros de eventos e mecanismos de alerta permite identificar situações que exigem intervenção administrativa.

Embora o TechHelpSolutions ainda não possua uma infraestrutura de produção implantada e monitorada continuamente, sua arquitetura e seus componentes permitem planejar essa implantação de maneira organizada.

Assim, a colocação em produção representa uma etapa posterior ao desenvolvimento acadêmico, exigindo a adequação das configurações de segurança, a definição de procedimentos operacionais e a implementação de mecanismos de acompanhamento para garantir a continuidade do serviço.

12. EVOLUÇÃO DO PROJETO

A evolução de um sistema consiste no planejamento e na implementação de melhorias destinadas a ampliar suas funcionalidades, corrigir limitações e adequar sua estrutura a novas necessidades.

O TechHelpSolutions foi desenvolvido como uma aplicação acadêmica funcional, contemplando recursos de gerenciamento de chamados, autenticação, controle de acesso e integração entre frontend, API REST e banco de dados SQL Server.

Embora os principais requisitos do projeto tenham sido implementados, foram identificadas oportunidades de melhoria relacionadas à segurança, ao desempenho, à experiência dos usuários e à preparação da aplicação para ambientes de produção.

12.1. Limitações Atuais

A versão atual do TechHelpSolutions atende aos objetivos estabelecidos para o desenvolvimento acadêmico, mas apresenta limitações que devem ser consideradas em uma possível utilização comercial.

Uma das principais limitações está relacionada à infraestrutura. O sistema foi desenvolvido e testado em ambiente local, sem uma implantação definitiva em servidor de produção ou mecanismos contínuos de monitoramento e disponibilidade.

Em relação à segurança, algumas configurações utilizadas durante o desenvolvimento precisam ser aprimoradas, incluindo o gerenciamento da chave JWT, as permissões de CORS e o armazenamento dos tokens de autenticação no navegador.

Também não foi implementado um mecanismo de revogação imediata dos tokens já emitidos quando ocorre a alteração da senha de um usuário.

Quanto ao banco de dados, os testes de desempenho foram realizados com um volume reduzido de registros. Embora tenham sido criados índices adicionais e analisados os planos de execução, ainda são necessários testes com bases maiores para avaliar o comportamento das consultas em situações de uso mais intenso.

As principais entidades do sistema possuem operações de cadastro, consulta, atualização e exclusão disponibilizadas pela API REST. Entretanto, nem todas essas operações estão presentes na interface web, pois determinadas funcionalidades administrativas foram disponibilizadas exclusivamente por meio dos endpoints da API, com acesso restrito ao perfil de administrador.

Esses aspectos não impedem a demonstração das funcionalidades desenvolvidas, mas representam pontos que exigem atenção em uma futura expansão do projeto.

12.2. Propostas de Melhoria

As propostas de evolução do TechHelpSolutions foram definidas considerando as limitações identificadas e as necessidades de uma aplicação destinada ao gerenciamento contínuo de chamados.

Uma das melhorias prioritárias consiste no fortalecimento dos mecanismos de segurança. Essa etapa inclui a utilização de configurações protegidas para informações sensíveis, a obrigatoriedade de HTTPS, a restrição das origens autorizadas e a implementação de estratégias de renovação e revogação de tokens.

Outra proposta envolve o aprimoramento da interface web, permitindo que funcionalidades administrativas já disponibilizadas pela API REST possam ser acessadas diretamente pelo frontend, conforme as necessidades dos usuários e as permissões estabelecidas para cada perfil.

No banco de dados, recomenda-se ampliar os testes de desempenho utilizando um volume maior de registros, de modo a avaliar a efetividade dos índices implementados e identificar oportunidades de otimização das consultas.

Também poderá ser considerada a implementação de mecanismos de auditoria para registrar alterações relevantes, permitindo identificar o responsável, a data e o tipo de operação realizada.

Em relação à experiência dos usuários, poderão ser desenvolvidos recursos adicionais, como filtros avançados de chamados, indicadores de atendimento e notificações sobre mudanças de situação.

Por fim, a implantação em um ambiente de produção deverá contemplar procedimentos automatizados de publicação, rotinas de backup, monitoramento de serviços e registro centralizado de erros.

12.3. Cronograma de Evolução

A evolução do TechHelpSolutions poderá ser organizada em quatro etapas, considerando a prioridade das melhorias e suas dependências técnicas.

Etapa	Melhorias previstas	Prazo estimado
1	Revisão das configurações de segurança, proteção de credenciais, HTTPS e gerenciamento de tokens.	1º mês
2	Integração de funcionalidades administrativas adicionais ao frontend e aprimoramento da interface web.	2º mês
3	Testes de desempenho com maior volume de dados, revisão dos índices e implementação de recursos de auditoria.	3º mês
4	Preparação da infraestrutura de produção, implantação, backup e monitoramento.	4º mês

Os prazos apresentados representam uma estimativa de planejamento e poderão ser ajustados conforme a disponibilidade da equipe, os recursos necessários e os resultados obtidos durante as etapas anteriores.

A organização proposta prioriza inicialmente a segurança da aplicação, seguida pela ampliação das funcionalidades, pela avaliação do desempenho e pela preparação do ambiente operacional.

Dessa forma, o planejamento de evolução demonstra que o TechHelpSolutions possui possibilidades de expansão além dos requisitos acadêmicos, permitindo seu aprimoramento gradual para atender a cenários de utilização mais abrangentes.

13. CONCLUSÃO

O desenvolvimento do TechHelpSolutions permitiu aplicar, de maneira integrada, os conhecimentos relacionados ao gerenciamento de bancos de dados, à programação avançada em T-SQL e ao desenvolvimento de aplicações web.

A solução desenvolvida contempla uma arquitetura composta por frontend, API REST e banco de dados Microsoft SQL Server, possibilitando o gerenciamento de chamados de suporte técnico por usuários com diferentes perfis de acesso.

No banco de dados, foram implementadas Stored Procedures para controlar a atribuição e a finalização de chamados, Functions para realizar cálculos reutilizáveis, Views para consolidar informações e Triggers para reforçar a integridade dos registros. Também foram utilizados mecanismos de transação e controle de concorrência, contribuindo para a consistência das operações realizadas.

A criação de índices e a análise dos planos de execução permitiram avaliar o comportamento das consultas e compreender a importância da utilização de métricas para verificar possíveis melhorias de desempenho. Os resultados obtidos demonstraram que a implementação de índices não garante, isoladamente, a redução do custo das consultas, sendo necessária a análise das características e do volume de dados.

Outro aspecto relevante foi a automatização da configuração do banco de dados por meio de um script PowerShell, facilitando a criação da estrutura, a aplicação dos objetos T-SQL e a preparação dos registros necessários aos testes.

A integração entre o SQL Server e a API desenvolvida com ASP.NET Core 9 demonstrou a aplicação prática dos recursos de persistência e das regras de negócio em um sistema funcional. A interface web, por sua vez, possibilitou a utilização dessas funcionalidades por meio de operações acessíveis aos diferentes perfis de usuários.

Em relação à segurança, foram implementados mecanismos de autenticação JWT, autorização baseada em perfis, proteção de senhas e parametrização de comandos SQL. Também foram identificadas limitações que deverão ser corrigidas antes de uma eventual implantação em ambiente de produção.

Conclui-se que o TechHelpSolutions atingiu os objetivos propostos para o desenvolvimento acadêmico, demonstrando a aplicação dos principais conceitos de Banco de Dados Avançado em uma solução integrada. Além de consolidar os conhecimentos técnicos envolvidos, o projeto apresenta possibilidades de evolução relacionadas à segurança, ao desempenho, à ampliação das funcionalidades e à preparação de uma infraestrutura de produção.

REFERÊNCIAS

MICROSOFT. **Documentação técnica do SQL Server**. Microsoft Learn. Disponível em: https://learn.microsoft.com/pt-br/sql/sql-server/?view=sql-server-ver17. Acesso em: 22 set. 2026.

MICROSOFT. **Criar APIs Web com o ASP.NET Core**. Microsoft Learn. Disponível em: https://learn.microsoft.com/pt-br/aspnet/core/web-api/?view=aspnetcore-10.0. Acesso em: 22 set. 2026.

MICROSOFT. **Provedor de Banco de Dados do Microsoft SQL Server – EF Core**. Microsoft Learn. Disponível em: https://learn.microsoft.com/pt-br/ef/core/providers/sql-server/?tabs=dotnet-core-cli%2Csqlserver. Acesso em: 22 set. 2026.